Main1 Extension: Hubbard-Holstein Mathematical Implementation (Substitution-First)

Jake Skyler Strobel (extension drafted with repo-grounded implementation mapping)

March 1, 2026

Contents

1 1. Parameter Manifest and Reader Contract **3**
1.1 1.1 Required parameter manifest . . . 3
1.2 1.2 Reader contract . . . 4
1.3 1.3 Conventions inherited from repository rules . . . 4
1.3.1 1.3.1 Internal Pauli alphabet . . . 4
1.3.2 1.3.2 Pauli-word and qubit ordering . . . 4
1.3.3 1.3.3 Number operator source of truth . . . 4
1.3.4 1.3.4 Canonical code anchors . . . 4
1.4 1.4 Final fully substituted manifest form . . . 4

2 2. Continuity Recap from main (1).pdf **4**
2.1 2.1 Primitive Hamiltonian recap . . . 4
2.2 2.2 VQE recap . . . 5
2.3 2.4 Continuity-to-extension map . . . 5
2.4 2.5 Final fully substituted continuity statement . . . 5

3 3. Ordering and Indexing Foundations **5**
3.1 3.1 Primitive index sets . . . 5
3.2 3.2 Two fermion ordering conventions . . . 5
3.2.1 3.2.1 Interleaved . . . 5
3.3 3.3 Pauli string placement map . . . 5
3.4 3.4 Basis-index extraction . . . 6
3.5 3.5 Explicit substitution examples . . . 6
3.5.1 3.5.1 Example A: interleaved, L=3 . . . 6
3.6 3.6 Final fully substituted ordering identity . . . 6
3.7 4.1 Primitive boson symbols . . . 6
3.8 4.3 Unary encoding . . . 6
3.8.1 4.4.1 Binary vacuum . . . 7
3.8.2 4.4.2 Unary vacuum . . . 7
3.9 4.5 Embedded local operator decomposition . . . 7
3.10 4.6 Final fully substituted boson register map . . . 7

4 5. Primitive Operators and Direct Substitution **7**
4.1 5.1 Fermionic ladder primitives . . . 7
4.2 5.3 Boson primitives . . . 7
4.3 5.4 Unary explicit forms . . . 8
4.4 5.5 Composite density primitives . . . 8
4.5 5.6 Final fully substituted primitive stack . . . 8
4.6 6.1 Primitive HH decomposition . . . 8
4.7 6.2 Substitution of fermion terms . . . 8

4.7.1 6.2.1 Substitute $\hat{n}_{i\sigma}$ . . . 8
4.7.2 6.2.2 Substitute onsite interaction . . . 9
4.7.3 6.3.1 Phonon energy . . . 9
4.8 6.4 Substitution of drive term . . . 9
4.9 6.5 Full substituted HH expression . . . 10

5 7. State, Expectation, and Exponential Primitives 10
5.1 7.1 State representation . . . 10
5.2 7.3 Expectation primitive . . . 10
5.3 7.4 Exponential primitive . . . 10
5.4 7.5 Final fully substituted state-update chain . . . 11

6 8. Inner Optimizer Deep Dive 11
6.1 8.1 VQE objective and data flow . . . 11
6.2 8.2 Restart objective decomposition . . . 11
6.3 8.3 SPSA branch (primary inner kernel) . . . 11
6.4 8.4 Legacy deterministic fallback branch (non-SPSA) . . . 12
6.5 8.5 ADAPT-linked reoptimization inside the inner loop (SPSA-main) . . . 12
6.6 8.6 Score map and repeat policy . . . 12
6.7 8.7 Finite-angle fallback as ADAPT pre-stop test . . . 12
6.8 8.8 HH seed preconditioning block . . . 13
6.9 8.9 Final fully substituted stopping map . . . 13
6.10 8.10 Deterministic-restart interpretation . . . 13
6.11 8.11 ADAPT inner-state recursion . . . 13
6.12 8.12 NFEV accounting . . . 14
6.13 8.13 Inner-loop objective geometry . . . 14
6.14 8.14 Restarts and deterministic invariants . . . 14
6.15 8.15 Commutator gradient in full tensor form . . . 14
6.16 8.16 Score shaping when repeats are allowed . . . 15
6.17 8.17 Finite-angle fallback as bounded two-point search . . . 15
6.18 8.18 Outer-loop stopping as a composite predicate . . . 15
6.19 8.19 Master ADAPT/VQE substitution chain (pool-generic) . . . 15
6.20 8.20 Primitive Pauli action substitution (bit-mask form) . . . 16
6.21 8.21 Primitive-closed energy functional . . . 16
6.22 8.22 Primitive-closed ADAPT gradient and update recursion . . . 17
6.23 8.23 Pool-construction case split and pipeline closure . . . 17
6.24 9.1 Cross-reference contract . . . 18
6.25 9.2 Minimal ADAPT recurrence (implementation form) . . . 18
6.26 9.3 Pool-family addendum . . . 18
6.27 9.4 Code-anchor index for Chapter 8 formulas . . . 18
6.28 10.1 HH termwise ansatz primitive . . . 19
6.29 10.2 HH layerwise ansatz primitive . . . 19
6.30 10.3 Substitution of grouped generators . . . 19
6.31 10.6 Final fully substituted HH-VA forms . . . 20
6.32 11.1 Primitive PAOP ingredients . . . 20
6.33 11.2 Core PI quadrature definition . . . 20
6.34 11.3 Shifted density substitution . . . 20
6.34.1 11.4.1 Local conditional displacement dressing . . . 20
6.34.2 11.4.6 Extended cloud channels . . . 21
6.35 11.6 Cleaning, normalization, split, dedup substitution . . . 21
6.36 11.7 Final fully substituted PAOP/PI operator-pool statement . . . 21

7 12. Appendix A: Code-Anchor Mathematical Ledger 21
7.1 12.1 HH construction ledger . . . 21
7.2 12.2 VQE and exponential ledger . . . 21
7.3 12.3 ADAPT ledger . . . 22
7.4 12.4 PAOP ledger . . . 22

7.5 B.1 Canonical PAOP templates . . . 22
7.6 B.2 Worked example A: nearest-neighbor hopdrag . . . 23
7.7 B.3 Worked example B: periodic edge hopdrag . . . 23
7.8 B.4 Signatures and dedup . . . 23
7.9 B.5 Final compact closure . . . 23

8 14. Appendix C: End-to-End Substitution Chain (Drive, Sector, Pool) 23
8.1 C.1 Variational functional and ansatz . . . 23
8.2 C.2 Hamiltonian decomposition after all substitutions . . . 23
8.3 C.3 Primitive substitutions . . . 24
8.4 C.4 ADAPT coupling and exact-sector target . . . 24
8.5 C.5 Final chain closure . . . 24

9 15. Appendix D: Alternate Inner Optimizer and Backend Branches 24
9.1 D.1 COBYLA alternate branch . . . 24
9.2 D.2 Compiled one-apply backend branch . . . 24
9.3 D.3 Optimizer/backend switch map . . . 25

1 1. Parameter Manifest and Reader Contract

This manuscript is a self-contained extension of `main (1).pdf`, with strict substitution-first derivations and direct mapping to repository implementation.

1.1 1.1 Required parameter manifest

- Model family/name: `Hubbard` (repo manifest convention); body scope explicitly extends to Hubbard-Holstein (HH).
- Ansatz types covered in this extension:
- Conventional VQE: Hubbard termwise/layerwise, HH termwise/layerwise, UCCSD variants.
- ADAPT-VQE: UCCSD pool, CSE pool, full-Hamiltonian pool, HH-HVA pool, PAOP pool families (`paop_min`, `paop_std`, `paop_full`, `paop_lf_std`, `paop_lf2_std`, `paop_lf_full`).
- Drive enabled: mathematically included in this document; optimizer and pool deep-dives are static-HH by default unless stated.
- Core physical parameters:
- Hopping: `t` (or `J` in HH constructor notation).
- Onsite interaction: `U`.
- Site potential offset: `dv`.
- HH-defining parameters:
- Phonon frequency: `omega0`.
- Electron-phonon coupling: `g` (repo arg name `g_ep` in pipeline CLI).
- Local phonon cutoff: `n_ph_max`.
- Boson encoding: `binary` and `unary` (implementation supports both in core HH builders).
- Reproducibility-defining implementation parameters discussed in this manuscript:
- ordering in fermion sector: `blocked` or `interleaved`.
- boundary: `open` or `periodic`.
- VQE optimizer method: includes `SPSA` (noise-oriented) plus SciPy methods (`COBYLA`, `SLSQP`).
- VQE energy backend: `legacy` or `one_apply_compiled`.
- VQE restart count, max iterations, initial point scale, bounds.
- ADAPT inner optimizer: `COBYLA` or `SPSA`.
- ADAPT SPSA controls: `adapt_spsa_a`, `adapt_spsa_c`, `adapt_spsa_alpha`, `adapt_spsa_gamma`, `adapt_spsa_A`, `adapt_spsa_avg_last`, `adapt_spsa_eval_repeats`, `adapt_spsa_eval_agg`.
- ADAPT state backend: `legacy` or `compiled`.
- ADAPT controls: `adapt_max_depth`, `adapt_eps_grad`, `adapt_eps_energy`, `adapt_maxiter`, repeat policy, finite-angle fallback settings, optional `adapt_gradient_parity_check`.
- PAOP controls: radius `paop_r`, split mode, prune threshold, normalization mode.

1.2 1.2 Reader contract

This document intentionally uses a linear symbolic style:

1. Primitive symbols are declared first.
2. Composite operators are assembled from primitives.
3. Substitutions are performed explicitly into expanded operator forms.
4. Final fully substituted forms are provided.
5. Every major block is mapped to implementation anchors.

1.3 1.3 Conventions inherited from repository rules

1.3.1 1.3.1 Internal Pauli alphabet

Internal Pauli letters are always:

$$\{e, x, y, z\}$$

with `e` as identity. Conversion to `I/X/Y/Z` is a boundary concern.

1.3.2 1.3.2 Pauli-word and qubit ordering

Pauli word strings are read left-to-right as:

$$q_{n-1} \cdots q_1 q_0$$

and qubit `q_0` is the rightmost character.

1.3.3 1.3.3 Number operator source of truth

The number operator is used as:

$$\hat{n}_p = \frac{I - Z_p}{2}$$

with rightmost-character indexing respected in string placement.

1.3.4 1.3.4 Canonical code anchors

- HH construction and mapping: `/Users/jakestrobel/Documents/Holstein_implementation/Holstein_test/src/quant`
- VQE and ansatz mechanics: `/Users/jakestrobel/Documents/Holstein_implementation/Holstein_test/src/quantum/`
- ADAPT pipeline and optimizer loop: `/Users/jakestrobel/Documents/Holstein_implementation/Holstein_test/pipe`
- PAOP pool families: `/Users/jakestrobel/Documents/Holstein_implementation/Holstein_test/src/quantum/opera`
- HF and HH reference state: `/Users/jakestrobel/Documents/Holstein_implementation/Holstein_test/src/quantum/`
- JW ladder primitives: `/Users/jakestrobel/Documents/Holstein_implementation/Holstein_test/src/quantum/paul`

1.4 1.4 Final fully substituted manifest form

The report-level reproducibility tuple used throughout is:

$$\mathcal{P} = (L, t, U, dv, \omega_0, g, n_{\mathrm{ph,max}}, \text{encoding}, \text{ordering}, \text{boundary}, \text{optimizer}, \text{restarts}, \text{maxiter}, \text{ADAPT controls}, \text{PAOP controls}, \text{drive}$$

with explicit implementation names in the listed source files. For current repo branches this tuple is refined to include

$$\mathcal{P}_{\mathrm{opt}} = (\texttt{vqe_method}, \texttt{vqe_energy_backend}, \texttt{adapt_inner_optimizer}, \texttt{adapt_state_backend}, \texttt{adapt_spsa_*}, \texttt{adapt_gradient_}$$

2 2. Continuity Recap from main (1).pdf

2.1 2.1 Primitive Hamiltonian recap

The baseline driven Hubbard form used in your main document is:

$$\hat{H}_{\mathrm{Hub}}(t) = -t \sum_{\langle i,j \rangle, \sigma} \left(\hat{c}^\dagger_{i\sigma} \hat{c}_{j\sigma} + \hat{c}^\dagger_{j\sigma} \hat{c}_{i\sigma} \right) + U \sum_i \hat{n}_{i\uparrow} \hat{n}_{i\downarrow} + \sum_{i,\sigma} (v_i(t) - v_i^{(0)}) \hat{n}_{i\sigma}.$$

Your style preference is to proceed from primitives to direct substitutions.

2.2 2.2 VQE recap

The VQE objective is:

$$E(\vec{\theta}) = \langle\psi(\vec{\theta})|\hat{H}|\psi(\vec{\theta})\rangle = \sum_j h_j \langle\psi(\vec{\theta})|\hat{P}_j|\psi(\vec{\theta})\rangle$$

with

$$|\psi(\vec{\theta})\rangle = U_p(\theta_p)\cdots U_1(\theta_1)|\psi_{\text{ref}}\rangle.$$

2.3 ADAPT recap

ADAPT selection signal in your notation is commutator-based:

$$g_m^{(n)} = i\langle\psi^{(n)}|[\hat{H}, A_m]|\psi^{(n)}\rangle.$$

This extension formalizes the exact implementation variant in the repo and extends it to HH pools, HH-VA, and PAOP momentum-quadrature mechanics.

2.3 2.4 Continuity-to-extension map

- Your Sections 1 through 3 are treated as prerequisites.
- This extension fills missing math for HH builders, optimizer internals, ADAPT pool realizations, HH-VA decomposition, and PAOP/PI pool construction.

2.4 2.5 Final fully substituted continuity statement

Given your prior Hubbard-only symbolic layer and drive notation, this extension substitutes:

$$\hat{H}_{\text{Hub}}(t) \mapsto \hat{H}_{\text{HH}}(t) = \hat{H}_t + \hat{H}_U + \hat{H}_v + \hat{H}_{\text{ph}} + \hat{H}_g + \hat{H}_{\text{drive}}$$

and then substitutes each constituent term to operator-level expressions on the full fermion-plus-phonon qubit register.

3 3. Ordering and Indexing Foundations

3.1 3.1 Primitive index sets

- Site index: $i \in \{0, 1, \ldots, L-1\}$.
- Spin label: $\sigma \in \{\uparrow, \downarrow\}$, encoded in code as `0` (up) and `1` (down).
- Fermion mode index: $p(i, \sigma)$.
- Qubit labels follow binary basis indexing where qubit 0 is least significant in basis index arithmetic.

3.2 3.2 Two fermion ordering conventions

3.2.1 3.2.1 Interleaved

$$p(i, \sigma) = 2i + \sigma, \quad \sigma \in \{0, 1\}.$$

3.2.2 Blocked

$$p(i, \uparrow) = i, \qquad p(i, \downarrow) = L + i.$$

Implementation anchor: `mode_index(...)` in `hubbard_latex_python_pairs.py`.

3.3 3.3 Pauli string placement map

Given any qubit index `q` and total qubits `nq`, the Pauli letter acting on `q` is placed at string position:

$$\text{pos}(q) = nq - 1 - q.$$

So for `z` on qubit `q`, the Pauli word is:

$$\text{word} = e^{\otimes(nq-1-q)}\, z\, e^{\otimes q}$$

when written left-to-right as `q_(nq-1)...q_0`.

3.4 3.4 Basis-index extraction

For basis index `k`, occupation bit on qubit `q` is:

$$b_q(k) = \left(\left\lfloor \frac{k}{2^q} \right\rfloor \bmod 2 \right) = ((k \gg q)\&1).$$

This is the exact bit extraction used by occupancy and doublon observables.

3.5 3.5 Explicit substitution examples

3.5.1 3.5.1 Example A: interleaved, L=3

$$\begin{aligned} p(0,\uparrow) = 0,\ p(0,\downarrow) = 1, \\ p(1,\uparrow) = 2,\ p(1,\downarrow) = 3, \\ p(2,\uparrow) = 4,\ p(2,\downarrow) = 5. \end{aligned}$$

3.5.2 Example B: blocked, L=3

$$\begin{aligned} p(0,\uparrow) = 0,\ p(1,\uparrow) = 1,\ p(2,\uparrow) = 2, \\ p(0,\downarrow) = 3,\ p(1,\downarrow) = 4,\ p(2,\downarrow) = 5. \end{aligned}$$

3.5.3 Example C: z placement for nq=8 and q=2

$$\mathrm{pos}(2) = 8 - 1 - 2 = 5$$

so word is `eeeeezee` where rightmost char is qubit 0.

3.6 3.6 Final fully substituted ordering identity

All indexing-sensitive formulas in this extension can be reduced to two substitutions:

$$(i,\sigma) \xrightarrow{\text{ordering}} p(i,\sigma) \xrightarrow{\text{word placement}} \mathrm{pos}(p) = nq - 1 - p.$$

4. Boson Register and Encodings

3.7 4.1 Primitive boson symbols

- Local cutoff: $n_{\mathrm{ph,max}}$.
- Local Hilbert dimension: $d = n_{\mathrm{ph,max}} + 1$.
- Boson qubits per site: q_{pb}.
- Global HH qubit count:

  $$N_q = 2L + L\, q_{\mathrm{pb}}.$$

  ## 4.2 Binary encoding

  $$q_{\mathrm{pb}} = \max\{1, \lceil \log_2(d) \rceil\}.$$

  Site `i` local boson block starts at:

  $$q_{\mathrm{base}}(i) = 2L + i\, q_{\mathrm{pb}}.$$

  Local bits are then `q_base(i),...,q_base(i)+q_pb-1` in increasing qubit index.

3.8 4.3 Unary encoding

$$q_{\mathrm{pb}} = d = n_{\mathrm{ph,max}} + 1.$$

One-hot local level `n` occupies qubit:

$$q(i,n) = 2L + i(n_{\mathrm{ph,max}} + 1) + n.$$

4.4 Boson vacuum substitution

3.8.1 4.4.1 Binary vacuum

All boson bits are `0`.

3.8.2 4.4.2 Unary vacuum

Per site: level `n=0` qubit set to `1`, others `0`.

Implementation anchor: `_phonon_vacuum_bitstring(...)` and `hubbard_holstein_reference_state(...)` in `hartree_fock_reference_state.py`.

3.9 4.5 Embedded local operator decomposition

In binary mode, local truncated matrices are embedded into `2^{q_pb}` and decomposed into local Pauli basis:

$$M = \sum_{\alpha \in \{I,X,Y,Z\}^{\otimes q_{pb}}} c_\alpha \, \alpha.$$

Then each local Pauli monomial is lifted to global word by qubit placement map.

In unary mode, operators are constructed directly by analytic Pauli formulas (no local dense decomposition needed).

3.10 4.6 Final fully substituted boson register map

$$\text{global register} = [\text{fermion qubits } 0 \ldots 2L-1 \mid \text{phonon qubits } 2L \ldots N_q - 1].$$

Every HH term substitution in later sections uses this block layout explicitly.

4 5. Primitive Operators and Direct Substitution

4.1 5.1 Fermionic ladder primitives

Using JW (source of truth in `pauli_polynomial_class.py`), creation and annihilation on mode `j` are represented as PauliPolynomial objects generated by `fermion_plus_operator("JW", nq, j)` and `fermion_minus_operator("JW", nq, j)`.

Symbolically:

$$\hat{c}_j^\dagger = \frac{1}{2}(X_j - iY_j)\prod_{k<j} Z_k, \qquad \hat{c}_j = \frac{1}{2}(X_j + iY_j)\prod_{k<j} Z_k.$$

5.2 Number primitive

Primitive definition:

$$\hat{n}_p = \hat{c}_p^\dagger \hat{c}_p.$$

Substitution:

$$\hat{n}_p = \frac{I - Z_p}{2}.$$

Word-placement substitution at total width `nq`:

$$Z_p \to \text{word with } z \text{ at index } nq - 1 - p.$$

Implementation anchor: `jw_number_operator(...)` in both HH and VQE modules.

4.2 5.3 Boson primitives

- annihilation: $\hat{b}_i$
- creation: $\hat{b}_i^\dagger$
- displacement: $\hat{x}_i = \hat{b}_i + \hat{b}_i^\dagger$
- momentum quadrature (PI in PAOP context):

$$\hat{P}_i = i(\hat{b}_i^\dagger - \hat{b}_i).$$

  Implementation anchor for `x`: `boson_displacement_operator(...)`. Implementation anchor for `P`: explicitly assembled in `polaron_paop.py` as `(1j * bdag_op) + (-1j * b_op)`.

4.3 5.4 Unary explicit forms

Unary number on site block qubits $q(i,0),...,q(i,N_b)$:

$$\hat{n}_{b,i} = \sum_{n=0}^{N_b} n\,\hat{n}_{i,n}, \qquad \hat{n}_{i,n} = \frac{I - Z_{q(i,n)}}{2}.$$

Unary displacement:

$$\hat{x}_i = \sum_{n=0}^{N_b-1} \sqrt{n+1}\,\frac{XX_{n,n+1} + YY_{n,n+1}}{2}.$$

These are directly implemented in `boson_unary_number_operator(...)` and `boson_unary_displacement_operator(...)`.

4.4 5.5 Composite density primitives

$$\hat{n}_i = \hat{n}_{i\uparrow} + \hat{n}_{i\downarrow}, \qquad \hat{D}_i = \hat{n}_{i\uparrow}\hat{n}_{i\downarrow}.$$

PAOP shifted density used in code:

$$\hat{n}_i - \bar{n}, \qquad \bar{n} = \frac{N_\uparrow + N_\downarrow}{L}$$

with fallback $\bar{n} = 1$ if needed.

4.5 5.6 Final fully substituted primitive stack

$$\left\{\hat{c}, \hat{c}^\dagger, \hat{n}_p, \hat{b}, \hat{b}^\dagger, \hat{x}, \hat{P}, \hat{n}_i, \hat{D}_i\right\} \xrightarrow{\text{JW + encoding + placement}} \sum_\ell c_\ell\,\hat{P}_\ell, \quad \hat{P}_\ell \in \{e,x,y,z\}^{\otimes N_q}.$$

6. Full Hubbard-Holstein Hamiltonian Derivation

4.6 6.1 Primitive HH decomposition

$$\hat{H}_{\text{HH}}(t) = \hat{H}_t + \hat{H}_U + \hat{H}_v + \hat{H}_{\text{ph}} + \hat{H}_g + \hat{H}_{\text{drive}}.$$

where:

$$\hat{H}_t = -t \sum_{\langle i,j\rangle,\sigma} \left(\hat{c}^\dagger_{i\sigma}\hat{c}_{j\sigma} + \hat{c}^\dagger_{j\sigma}\hat{c}_{i\sigma}\right),$$

$$\hat{H}_U = U \sum_i \hat{n}_{i\uparrow}\hat{n}_{i\downarrow},$$

$$\hat{H}_v = -\sum_{i,\sigma} v_i \hat{n}_{i\sigma},$$

$$\hat{H}_{\text{ph}} = \omega_0 \sum_i \left(\hat{n}_{b,i} + \frac{1}{2}\right),$$

$$\hat{H}_g = g \sum_i \hat{x}_i(\hat{n}_i - I),$$

$$\hat{H}_{\text{drive}} = \sum_{i,\sigma} (v_i(t) - v_{0,i})\hat{n}_{i\sigma}.$$

Implementation anchor: `build_hubbard_holstein_hamiltonian(...)`.

4.7 6.2 Substitution of fermion terms

4.7.1 6.2.1 Substitute $\hat{n}_{i\sigma}$

$$\hat{n}_{i\sigma} = \frac{I - Z_{p(i,\sigma)}}{2}$$

with `p(i,sigma)` replaced by chosen ordering map.

4.7.2 6.2.2 Substitute onsite interaction

$$\hat{H}_U = U \sum_i \left(\frac{I - Z_{p(i,\uparrow)}}{2} \right) \left(\frac{I - Z_{p(i,\downarrow)}}{2} \right)$$

$$= \frac{U}{4} \sum_i \left(I - Z_{p(i,\uparrow)} - Z_{p(i,\downarrow)} + Z_{p(i,\uparrow)} Z_{p(i,\downarrow)} \right).$$

6.2.3 Substitute potential term

$$\hat{H}_v = -\sum_{i,\sigma} v_i \frac{I - Z_{p(i,\sigma)}}{2} = -\frac{1}{2} \sum_{i,\sigma} v_i I + \frac{1}{2} \sum_{i,\sigma} v_i Z_{p(i,\sigma)}.$$

6.3 Substitution of phonon terms

4.7.3 6.3.1 Phonon energy

$$\hat{H}_{\text{ph}} = \omega_0 \sum_i \hat{n}_{b,i} + \frac{L\omega_0}{2} I.$$

For unary substitution:

$$\hat{n}_{b,i} = \sum_{n=0}^{N_b} n \, \frac{I - Z_{q(i,n)}}{2}.$$

6.3.2 Electron-phonon coupling

$$\hat{H}_g = g \sum_i \hat{x}_i \left(\hat{n}_{i\uparrow} + \hat{n}_{i\downarrow} - I \right)$$

Substitute number operators:

$$\hat{n}_{i\uparrow} + \hat{n}_{i\downarrow} - I = \frac{I - Z_{p(i,\uparrow)}}{2} + \frac{I - Z_{p(i,\downarrow)}}{2} - I = -\frac{1}{2} \left(Z_{p(i,\uparrow)} + Z_{p(i,\downarrow)} \right).$$

So

$$\hat{H}_g = -\frac{g}{2} \sum_i \hat{x}_i \left(Z_{p(i,\uparrow)} + Z_{p(i,\downarrow)} \right).$$

This form is useful for direct Pauli-term expansion when $\hat{x}_i$ is already decomposed.

4.8 6.4 Substitution of drive term

$$\hat{H}_{\text{drive}} = \sum_{i,\sigma} (v_i(t) - v_{0,i}) \frac{I - Z_{p(i,\sigma)}}{2}$$

$$= \frac{1}{2} \sum_{i,\sigma} (v_i(t) - v_{0,i}) I - \frac{1}{2} \sum_{i,\sigma} (v_i(t) - v_{0,i}) Z_{p(i,\sigma)}.$$

In implementation, this is built by negating deltas and routing through the existing Hubbard potential builder.

4.9 6.5 Full substituted HH expression

Collecting previous substitutions:

$$\begin{aligned}\hat{H}_{\mathrm{HH}}(t) = &- t \sum_{\langle i,j\rangle,\sigma} \left(\hat{c}^\dagger_{i\sigma}\hat{c}_{j\sigma} + \hat{c}^\dagger_{j\sigma}\hat{c}_{i\sigma}\right) \\ &+ \frac{U}{4}\sum_i \left(I - Z_{p(i,\uparrow)} - Z_{p(i,\downarrow)} + Z_{p(i,\uparrow)}Z_{p(i,\downarrow)}\right) \\ &- \frac{1}{2}\sum_{i,\sigma} v_i I + \frac{1}{2}\sum_{i,\sigma} v_i Z_{p(i,\sigma)} \\ &+ \omega_0 \sum_i \hat{n}_{b,i} + \frac{L\omega_0}{2} I \\ &- \frac{g}{2}\sum_i \hat{x}_i \left(Z_{p(i,\uparrow)} + Z_{p(i,\downarrow)}\right) \\ &+ \frac{1}{2}\sum_{i,\sigma}(v_i(t) - v_{0,i})I - \frac{1}{2}\sum_{i,\sigma}(v_i(t) - v_{0,i})Z_{p(i,\sigma)}.\end{aligned}$$

Final final step in code is polynomial reduction into merged coefficients over unique `e/x/y/z` words.

5 7. State, Expectation, and Exponential Primitives

5.1 7.1 State representation

$$|\psi\rangle = \sum_{k=0}^{2^{N_q}-1} \psi_k |k\rangle, \qquad \sum_k |\psi_k|^2 = 1.$$

7.2 Pauli-word action

For Pauli word $P \in \{e, x, y, z\}^{\otimes N_q}$:

$$P|\psi\rangle \text{ is implemented by bit flips and phase multipliers per qubit letter.}$$

Implementation anchor: `apply_pauli_string(...)`.

5.2 7.3 Expectation primitive

Given

$$\hat{H} = \sum_j h_j P_j,$$

$$\langle H\rangle = \sum_j h_j \langle\psi|P_j|\psi\rangle.$$

Implementation anchor: `expval_pauli_polynomial(...)`.

5.3 7.4 Exponential primitive

For single Pauli $P^2 = I$:

$$\exp(-i\alpha P) = \cos(\alpha)I - i\sin(\alpha)P.$$

For PauliPolynomial $G = \sum_j c_j P_j$, implementation uses ordered first-order product:

$$\exp(-i\theta G)|\psi\rangle \approx \prod_j \exp(-i\theta c_j P_j)|\psi\rangle.$$

with optional lexical sorting of terms.

Implementation anchor: `apply_exp_pauli_polynomial(...)`.

5.4 7.5 Final fully substituted state-update chain

$$|\psi_{k+1}\rangle = \left(\prod_{m=1}^{M} \prod_{j \in \mathcal{T}_m} \exp\left(-i\theta_m c_{m,j} P_{m,j}\right) \right) |\psi_{\mathrm{ref}}\rangle$$

where each generator block m maps to an ansatz term or ADAPT-selected operator.

6 8. Inner Optimizer Deep Dive

6.1 8.1 VQE objective and data flow

For ansatz state

$$\psi(\theta) = \prod_{k=1}^{p} \exp(-i\theta_k A_k)\psi_{\mathrm{ref}},$$

the inner optimizer solves

$$E(\theta) = \langle \psi(\theta) | \hat{H} | \psi(\theta) \rangle, \qquad \theta \in \mathbb{R}^p,$$

with `p = ansatz.num_parameters`.

Implementation path in `vqe_minimize`:

1. build `energy_fn(x)` from `ansatz.prepare_state(x, psi_ref)` and `expval_pauli_polynomial`.
2. run restart count R times.
3. select the best final (E, theta) pair.

Code anchor: `src/quantum/vqe_latex_python_pairs.py` (function `vqe_minimize`).

6.2 8.2 Restart objective decomposition

For restart r,

$$\theta_0^{(r)} = \sigma\, \xi^{(r)}, \qquad \xi^{(r)} \sim \mathcal{N}(0, I_p), \quad \sigma = \texttt{initial_point_stddev}.$$

Seed RNG is deterministic because `seed` is fixed by call path.

Each restart returns

$$(\theta_*^{(r)}, E_*^{(r)}, \mathsf{nfev}^{(r)}, \mathsf{nit}^{(r)}, \mathsf{success}^{(r)}, \mathsf{message}^{(r)}),$$

and the selected restart is

$$r^* = \arg\min_r E_*^{(r)}, \qquad E^* = E_*^{(r^*)}, \qquad \theta^* = \theta_*^{(r^*)}.$$

6.3 8.3 SPSA branch (primary inner kernel)

For `method=SPSA`, each inner iteration uses perturbation pair evaluations:

$$\Delta_k \in \{-1, +1\}^p, \qquad \theta_k^{\pm} = \Pi_{\mathcal{B}}(\theta_k \pm c_k \Delta_k)\,,$$

$$y_k^{\pm} = \mathrm{Agg}\left(\left\{ E\left(\theta_k^{\pm}\right) \right\}_{s=1}^{R_{\mathrm{eval}}} \right), \qquad \mathrm{Agg} \in \{\mathrm{mean}, \mathrm{median}\}.$$

Gradient estimator:

$$\widehat{g}_k = \frac{y_k^+ - y_k^-}{2c_k}\, \Delta_k^{-1} = \frac{y_k^+ - y_k^-}{2c_k}\, \Delta_k,$$

since each component of Δ_k is ± 1. Gain schedules:

$$a_k = \frac{a}{(k+1+A)^{\alpha}}, \qquad c_k = \frac{c}{(k+1)^{\gamma}}.$$

Update:

$$\theta_{k+1} = \Pi_{\mathcal{B}}(\theta_k - a_k \widehat{g}_k)\,.$$

Optional terminal averaging (`avg_last = m`):

$$\bar{\theta}_T = \frac{1}{m}\sum_{j=0}^{m-1}\theta_{T-j}, \qquad \theta_\star = \begin{cases}\bar{\theta}_T, & m > 0,\\ \theta_T, & m = 0.\end{cases}$$

Implementation anchor: `spsa_minimize(...)` in `src/quantum/spsa_optimizer.py`, called via `vqe_minimize` and `_run_hardcoded_adapt_vqe`.

6.4 8.4 Legacy deterministic fallback branch (non-SPSA)

If SciPy is missing, fallback performs:

$$\text{for } t = 1,\ldots,\texttt{maxiter}, \quad \delta_t = \delta_{t-1}\times\mathbf{1}_{\exists\Delta E<0} + \frac{\delta_{t-1}}{2}\mathbf{1}_{\nexists\Delta E<0}, \text{proposals } \theta\pm\delta_t e_k,\ k = 1,\ldots,p, \text{ greedily accepted if energy decreases}$$

With $\delta_0 = 0.2$, step halving and acceptance rule, function eval count obeys

$$\mathsf{nfev}_{\mathrm{fb}} \le 1 + 2p\,\texttt{maxiter},$$

where the leading `1` is the start-energy evaluation and the `2p` term applies per scan round until step decay truncates at 10^{-6}.

6.5 8.5 ADAPT-linked reoptimization inside the inner loop (SPSA-main)

When ADAPT appends an operator, parameters are all re-optimized jointly:

$$\theta_n^* = \arg\min_{\theta\in\mathbb{R}^n} E_n(\theta), \qquad E_n(\theta) = \langle\psi_n(\theta)|\hat{H}|\psi_n(\theta)\rangle.$$

In primary flow this is executed by SPSA as

$$\theta_n^* \approx \mathrm{SPSA}\Big(E_n, \theta_n^{(0)};\ a, c, \alpha, \gamma, A, R_{\mathrm{eval}}, \mathrm{Agg}, m\Big).$$

where $\theta_n^{(0)}$ is the appended vector before reoptimization. This happens every ADAPT iteration where a new operator is appended. COBYLA branch is documented in Appendix D.1.

6.6 8.6 Score map and repeat policy

Pool gradient magnitudes are

$$g_m^{(n)} = i\langle\psi_n|[\hat{H}, A_m]|\psi_n\rangle, \qquad m \in \mathcal{I}_n.$$

If `allow_repeats=True`, implementation uses diversity-biased scoring:

$$\mathrm{score}_m = \frac{|g_m^{(n)}|}{1 + 1.5\,c_m},$$

where c_m counts how many times index m was already selected. For no-repeat mode, $\mathrm{score}_m = |g_m^{(n)}|$.

6.7 8.7 Finite-angle fallback as ADAPT pre-stop test

If $\max_m |g_m^{(n)}| < \epsilon_{\mathrm{grad}}$, optional rescue tests every available index $m \in \mathcal{I}_n$ and $s \in \{\pm\theta_{\mathrm{fa}}\}$:

$$E_{m,s}^{(n)} = E_n(\theta \oplus s).$$

If

$$\min_{m,s} E_{m,s}^{(n)} \le E_n - \epsilon_{\mathrm{fa}}, \quad \epsilon_{\mathrm{fa}} := \texttt{finite_angle_min_improvement},$$

the selected pair is accepted and the loop continues from that fixed probe angle; otherwise stop reason is `eps_grad`.

6.8 8.8 HH seed preconditioning block

For HH (`problem="hh"`, $g_{\mathrm{ep}} \neq 0$, and seed not disabled), there is an extra pre-pass:

1. identify seed terms whose labels begin with `hh_termwise_ham_quadrature_term(`.
2. require at least one boson-Y and one electron-Z support in the Pauli polynomial.
3. optimize the seed vector in primary flow with SPSA:

$$\phi_* \approx \mathrm{SPSA}(E_{\mathrm{seed}}, \phi_0;\ a, c, \alpha, \gamma, A, R_{\mathrm{eval}}, \mathrm{Agg}, m)\,, \quad \texttt{maxiter} = \max(100, \min(\texttt{adapt_maxiter}, 600)).$$

The seeded state then becomes the initial state for the greedy loop. COBYLA seed-preopt branch is listed in Appendix D.1.

6.9 8.9 Final fully substituted stopping map

ADAPT outer exits when the stop predicate

$$\mathcal{T}_n = \mathbf{1}\left(\max_{m\in\mathcal{I}_n} |g_m^{(n)}| < \epsilon_{\mathrm{grad}}\right) + \mathbf{1}(|E_n - E_{n-1}| < \epsilon_{\mathrm{energy}}) + \mathbf{1}(n = \texttt{adapt_max_depth}) + \mathbf{1}(\texttt{allow_repeats=False} \wedge \mathcal{I}_n = \varnothing)$$

becomes true. With fallback disabled, the first term is the only gradient channel; with finite-angle rescue enabled, the first term becomes

$$\mathbf{1}\left(\max_{m\in\mathcal{I}_n} |g_m^{(n)}| < \epsilon_{\mathrm{grad}} \wedge \min_{m\in\mathcal{I}_n, s\in\{\pm\theta_{\mathrm{fa}}\}} E_{m,s}^{(n)} - E_n \geq -\epsilon_{\mathrm{fa}}\right).$$

Hence the overall map is

$$(\text{VQE restart-minimization}) + \big(\text{ADAPT gradient-greedy with full SPSA refit}\big) \to (E_{\mathrm{final}}, \theta_{\mathrm{final}}, \mathcal{S}_{\mathrm{final}}).$$

COBYLA variant is treated as alternate branch in Appendix D.1.

6.10 8.10 Deterministic-restart interpretation

For fixed backend parameters, each restart obeys

$$x_0^{(r)} = \sigma\,\xi_r, \qquad \xi_r = \texttt{RNG}_{\mathrm{seed}}(r) \in \mathbb{R}^p, \qquad E_{\mathrm{best}}^{(r)} = E(\theta_{\mathrm{opt}}^{(r)}),$$

and SPSA perturbations are

$$\Delta_k^{(r)} = \texttt{RNG}_{\mathrm{seed}}(r, k) \in \{-1, +1\}^p.$$

and therefore

$$\theta_{\mathrm{opt}}^{(r)}, E_{\mathrm{best}}^{(r)} \text{ are fixed functions of } r$$

inside a deterministic statevector backend with identical floating-point reduction order.

On noisy or hardware-like backends, measured objective has shot noise:

$$\tilde{E}(\theta) = E_{\mathrm{exp}}(\theta) + \eta, \qquad \mathbb{E}[\eta] = 0, \quad \eta \text{ depends on shots and hardware drift.}$$

There, repeat schedules may be justified even with identical seeds.

6.11 8.11 ADAPT inner-state recursion

With selected generator list $\mathcal{S}_n = [A_{m_1}, \ldots, A_{m_n}]$, each inner solve optimizes

$$E_n(\theta) = \left\langle \psi_{\mathrm{ref}} \middle| \prod_{k=n}^{1} \exp(i\theta_k A_{m_k})\, H \prod_{k=1}^{n} \exp(-i\theta_k A_{m_k}) \middle| \psi_{\mathrm{ref}} \right\rangle, \quad \theta \in \mathbb{R}^n.$$

If finite-angle rescue fails, ADAPT keeps

$$\theta_n^\star = \theta_{n-1}^\star, \qquad E_n^\star = E_{n-1}^\star$$

and exits on `eps_grad`.

6.12 8.12 NFEV accounting

Let $n_{\text{par}}^{(n)}$ be the ADAPT parameter count before re-optimization at iteration n, ν_n inner-optimizer objective count at iteration n, and S_n the fallback scan size. Then a useful accounting identity is

$$N_{\text{tot}}^{(N)} = \sum_{n=1}^{N} \nu_n + \sum_{n=1}^{N} \mathbf{1}_{\text{fallback}_n} S_n.$$

For SPSA inner optimization with eval repeats R_{eval} and iteration count T_n:

$$\nu_n \approx 2R_{\text{eval}} T_n, \qquad \nu_n \leq 2R_{\text{eval}}\texttt{adapt_maxiter}.$$

If averaging over last `avg_last` parameters is enabled, this modifies terminal parameter reporting but not objective-evaluation count.

6.13 8.13 Inner-loop objective geometry

For fixed selected sequence $\mathcal{S}_n = [A_{m_1}, \ldots, A_{m_n}]$ and parameter vector $\Theta_n = (\theta_1, \ldots, \theta_n)$, define

$$E_n(\Theta_n) = \langle \psi_{\text{ref}} | U^\dagger(\Theta_n; \mathcal{S}_n)\, H\, U(\Theta_n; \mathcal{S}_n) | \psi_{\text{ref}} \rangle, \qquad U(\Theta_n; \mathcal{S}_n) = \prod_{k=n}^{1} e^{-i\theta_k A_{m_k}},$$

where product order matches implementation call order in `_prepare_adapt_state`.

Energy differences logged by the inner optimizer are

$$\Delta E_n(\Theta_n, \Theta_n') = E_n(\Theta_n') - E_n(\Theta_n),$$

so the ADAPT-level recurrence is the nested optimization map

$$\theta_n^* = \arg\min_{\theta \in \mathbb{R}^n} E_n(\theta), \qquad \mathcal{E}_n = E_n(\theta_n^*) = \mathcal{E}_{n-1} + \Delta E_n(\theta_n^*, \theta_{n-1}^*),$$

with $\theta_n^* = (\theta_{n-1}^*, \theta_n)$ after append-and-refit.

6.14 8.14 Restarts and deterministic invariants

`vqe_minimize` enforces

$$\left(\Theta^{(r)}, \mathcal{E}^{(r)}\right) = \arg\min_{\Theta} \mathcal{J}_r(\Theta), \quad \mathcal{J}_r(\Theta) = \langle \psi_{\text{ref}} |\, U^\dagger(\Theta; \mathcal{S}_n)\, H\, U(\Theta; \mathcal{S}_n)\, | \psi_{\text{ref}} \rangle\,,$$

with fixed seed

$$\Theta_0^{(r)} = \sigma z_r, \qquad z_r \sim \mathcal{N}(0, I_n),$$

for $r = 1, \ldots, \texttt{restarts}$.

The selected restart pair is therefore

$$r^\star = \arg\min_r \mathcal{E}^{(r)}, \quad (E_n, \Theta_n) = \left(\mathcal{E}^{(r^\star)}, \Theta^{(r^\star)}\right).$$

For SPSA, each restart is deterministic for fixed seed because the perturbation stream $\{\Delta_k^{(r)}\}_k$ and objective backend are fixed.

6.15 8.15 Commutator gradient in full tensor form

For candidate operator index m, the exact selection rule used in code is

$$g_m^{(n)} = i\langle \psi_n | [H, A_m] | \psi_n \rangle,$$

implemented as

$$g_m^{(n)} = 2\,\Im\,\langle H\psi_n | A_m \psi_n \rangle$$

with

$$\psi_n = \left(\prod_{k=n}^{1} e^{-i\theta_k A_{m_k}} \right) |\psi_{\text{ref}}\rangle.$$

This form is equivalent to the commutator identity above and uses only one Hamiltonian action and one operator action per candidate.

6.16 8.16 Score shaping when repeats are allowed

When `allow_repeats=True`, score map is

$$\rho_m^{(n)} = \frac{|g_m^{(n)}|}{1 + \beta c_m^{(n)}}, \qquad \beta = 1.5.$$

The selected index is then

$$m_n = \arg\max_{m \in \mathcal{I}_n} \rho_m^{(n)},$$

where $\mathcal{I}_n$ is the available index set after masking unavailable entries.

If repeats are disabled, the mask is

$$\mathcal{I}_n = \{m : \ c_m^{(n)} = 0\},$$

and scoring reduces to raw magnitude.

6.17 8.17 Finite-angle fallback as bounded two-point search

When $\max_m |g_m^{(n)}| < \epsilon_{\text{grad}}$, the code tests

$$\{(m, s) : m \in \mathcal{I}_n, \ s \in \{-\theta_{\text{fa}}, +\theta_{\text{fa}}\}\},$$

then defines

$$(m^\star, s^\star) = \arg\min_{(m,s)} E_n(\theta \oplus s\, e_m)\,, \qquad \delta E^\star = E_n(\theta \oplus s^\star e_{m^\star}) - E_n(\theta).$$

The branch updates and possible exit are

$$\text{accept if} \quad -\delta E^\star > \epsilon_{\text{fa}}, \ (m_n, \theta_n) = (m^\star, s^\star), \qquad \text{else stop} = \text{eps_grad}.$$

6.18 8.18 Outer-loop stopping as a composite predicate

Let

$$\mathcal{C}_n^{\text{grad}}, \ \mathcal{C}_n^{\text{energy}}, \ \mathcal{C}_n^{\text{depth}}, \ \mathcal{C}_n^{\text{pool}}$$

denote the four boolean stop tests. Then implementation-level stopping is

$$\text{stop}_n = \mathcal{C}_n^{\text{grad}} \vee \mathcal{C}_n^{\text{energy}} \vee \mathcal{C}_n^{\text{depth}} \vee \mathcal{C}_n^{\text{pool}},$$

with

$$\mathcal{C}_n^{\text{depth}} = \{n = \texttt{adapt_max_depth}\}, \quad \mathcal{C}_n^{\text{pool}} = \{\neg\texttt{allow_repeats} \wedge \mathcal{I}_n = \varnothing\}.$$

The output object records this as `stop_reason` plus total objective-evaluation count.

6.19 8.19 Master ADAPT/VQE substitution chain (pool-generic)

Define problem label $\pi \in \{\text{hubbard}, \text{hh}\}$, pool key κ, and generic pool

$$\mathcal{P}^{(\kappa)} = \{A_m\}_{m=1}^{M_\kappa}, \qquad A_m = \sum_{u=1}^{U_m} a_{m,u} R_{m,u}, \qquad R_{m,u} \in \{e, x, y, z\}^{\otimes N_q}.$$

Hamiltonian primitive is

$$H = \sum_{r=1}^{N_H} h_r Q_r, \qquad Q_r \in \{e, x, y, z\}^{\otimes N_q}.$$

Reference state primitive is

$$|\psi_{\text{ref}}\rangle = \sum_{k=0}^{2^{N_q}-1} \psi_k^{\text{ref}} |k\rangle.$$

For selected indices $(m_1, \ldots, m_n)$ and angles $\Theta_n = (\theta_1, \ldots, \theta_n)$:

$$U_n(\Theta_n) = \prod_{j=1}^{n} e^{-i\theta_j A_{m_j}}, \qquad |\psi_n(\Theta_n)\rangle = U_n(\Theta_n)|\psi_{\text{ref}}\rangle.$$

Objective:

$$E_n(\Theta_n) = \langle\psi_n(\Theta_n)|H|\psi_n(\Theta_n)\rangle.$$

Substitute H:

$$E_n(\Theta_n) = \sum_{r=1}^{N_H} h_r \langle\psi_n(\Theta_n)|Q_r|\psi_n(\Theta_n)\rangle.$$

For fixed-ansatz VQE (`vqe_minimize`) this same $E(\Theta)$ is minimized over a fixed ansatz map; for ADAPT this is minimized after each append step with current selected list.

Implementation substitution for each appended generator (from `apply_exp_pauli_polynomial`) is

$$e^{-i\theta_j A_{m_j}} \rightsquigarrow \prod_{u=1}^{U_{m_j}} e^{-i\theta_j\, \Re(a_{m_j,u})\, R_{m_j,u}},$$

applied in deterministic sorted Pauli-term order.

6.20 8.20 Primitive Pauli action substitution (bit-mask form)

For any Pauli word $P \in \{e, x, y, z\}^{\otimes N_q}$ define:

$$\nu(P) = \sum_{q:\, P[q]\in\{x,y\}} 2^q, \qquad \chi_P(k) = \sum_{q:\, P[q]=z} b_q(k) + \sum_{q:\, P[q]=y} b_q(k), \qquad n_y(P) = |\{q : P[q] = y\}|.$$

Then

$$\omega_P(k) = i^{\,n_y(P)}(-1)^{\chi_P(k)}, \qquad P|k\rangle = \omega_P(k)\,|k \oplus \nu(P)\rangle.$$

Hence for state amplitudes $\alpha_k = \langle k|\psi\rangle$:

$$(P\psi)_k = \omega_P(k \oplus \nu(P))\, \alpha_{k\oplus\nu(P)}.$$

This is the primitive action used by `apply_pauli_string` and by polynomial accumulation in `_apply_pauli_polynomial`.

6.21 8.21 Primitive-closed energy functional

Write

$$|\psi_n(\Theta_n)\rangle = \sum_{k=0}^{2^{N_q}-1} \alpha_k^{(n)}(\Theta_n)|k\rangle.$$

Substitute Pauli action from 8.20 into 8.19:

$$\langle\psi_n|Q_r|\psi_n\rangle = \sum_{k=0}^{2^{N_q}-1} \left(\alpha_k^{(n)}\right)^* \omega_{Q_r}(k)\, \alpha_{k\oplus\nu(Q_r)}^{(n)}.$$

Therefore the fully substituted primitive form is

$$E_n(\Theta_n) = \sum_{r=1}^{N_H} h_r \sum_{k=0}^{2^{N_q}-1} \left(\alpha_k^{(n)}(\Theta_n)\right)^* \omega_{Q_r}(k)\, \alpha_{k\oplus\nu(Q_r)}^{(n)}(\Theta_n).$$

For fixed-ansatz restarts:

$$\Theta_{(r)}^\star = \arg\min_\Theta E(\Theta; \Theta_0^{(r)}), \qquad r^\star = \arg\min_r E(\Theta_{(r)}^\star), \qquad E_{\text{VQE}}^\star = E(\Theta_{(r^\star)}^\star).$$

6.22 8.22 Primitive-closed ADAPT gradient and update recursion

Commutator gradient:

$$g_m^{(n)} = i\langle\psi_n|[H, A_m]|\psi_n\rangle = i\sum_{r=1}^{N_H}\sum_{u=1}^{U_m} h_r a_{m,u}\langle\psi_n|[Q_r, R_{m,u}]|\psi_n\rangle.$$

Implementation-equivalent form:

$$g_m^{(n)} = 2\,\Im\langle H\psi_n|A_m\psi_n\rangle.$$

With 8.20 substitutions:

$$(H\psi_n)_k = \sum_{r=1}^{N_H} h_r\,\omega_{Q_r}(k\oplus\nu(Q_r))\,\alpha^{(n)}_{k\oplus\nu(Q_r)},$$

$$(A_m\psi_n)_k = \sum_{u=1}^{U_m} a_{m,u}\,\omega_{R_{m,u}}(k\oplus\nu(R_{m,u}))\,\alpha^{(n)}_{k\oplus\nu(R_{m,u})},$$

$$g_m^{(n)} = 2\,\Im\sum_{k=0}^{2^{N_q}-1}[(H\psi_n)_k]^*\,(A_m\psi_n)_k.$$

Selection score map:

$$s_m^{(n)} = \begin{cases} |g_m^{(n)}|, & \neg\texttt{allow_repeats}, \\ \dfrac{|g_m^{(n)}|}{1+\beta c_m^{(n)}}, & \texttt{allow_repeats},\ \beta = 1.5. \end{cases}$$

Index update:

$$m_n = \arg\max_{m\in\mathcal{I}_n} s_m^{(n)}.$$

Parameter append and inner optimization:

$$\Theta_{n+1}^{(0)} = (\Theta_n, \theta_{\text{init}}), \qquad \Theta_{n+1}^{\star} \approx \text{SPSA}\Big(E_{n+1}, \Theta_{n+1}^{(0)};\ a, c, \alpha, \gamma, A, R_{\text{eval}}, \text{Agg}, m\Big).$$

Pool-availability update:

$$\mathcal{I}_{n+1} = \begin{cases} \mathcal{I}_n \setminus \{m_n\}, & \neg\texttt{allow_repeats}, \\ \mathcal{I}_n, & \texttt{allow_repeats}. \end{cases}$$

Finite-angle fallback substitution:

$$(m^\star, s^\star) = \arg\min_{m\in\mathcal{I}_n,\ s\in\{\pm\theta_{\text{fa}}\}} E_{n+1}(\Theta_n \oplus s\,e_m),$$

$$\text{accept fallback} \iff E_n - E_{n+1}(\Theta_n \oplus s^\star e_{m^\star}) > \epsilon_{\text{fa}}.$$

Composite stopping predicate:

$$\text{Stop}_n = \mathbf{1}\Big(\max_{m\in\mathcal{I}_n}|g_m^{(n)}| < \epsilon_{\text{grad}}\ \wedge\ \text{no accepted fallback}\Big)\vee\mathbf{1}(|E_n - E_{n-1}| < \epsilon_{\text{energy}})\vee\mathbf{1}(n = \texttt{adapt_max_depth})\vee\mathbf{1}(\neg\texttt{allow_repea}$$

6.23 8.23 Pool-construction case split and pipeline closure

Pool construction map in `_run_hardcoded_adapt_vqe`:

$$\mathcal{K}_{\text{paop}} = \{\texttt{paop}, \texttt{paop_min}, \texttt{paop_std}, \texttt{paop_full}, \texttt{paop_lf}, \texttt{paop_lf_std}, \texttt{paop_lf2_std}, \texttt{paop_lf_full}\},$$

$$\mathcal{P}^{(\kappa)} = \begin{cases} \mathcal{P}_{\text{uccsd}}, & \pi = \text{hubbard},\ \kappa = \text{uccsd}, \\ \mathcal{P}_{\text{cse}}, & \pi = \text{hubbard},\ \kappa = \text{cse}, \\ \mathcal{P}_{\text{fullH}}, & \pi = \text{hubbard},\ \kappa = \text{full_hamiltonian}, \\ \mathcal{P}_{\text{hva}}, & \pi = \text{hh},\ \kappa = \text{hva},\ g_{\text{ep}} = 0, \\ \text{Dedup}(\mathcal{P}_{\text{hva}} \cup \mathcal{P}_{\text{hh_termwise}}), & \pi = \text{hh},\ \kappa = \text{hva},\ g_{\text{ep}} \neq 0, \\ \mathcal{P}_{\text{paop}}^{(\kappa)}, & \pi = \text{hh},\ \kappa \in \mathcal{K}_{\text{paop}},\ g_{\text{ep}} = 0, \\ \text{Dedup}\Big(\mathcal{P}_{\text{hva}} \cup \mathcal{P}_{\text{hh_termwise}} \cup \mathcal{P}_{\text{paop}}^{(\kappa)}\Big), & \pi = \text{hh},\ \kappa \in \mathcal{K}_{\text{paop}},\ g_{\text{ep}} \neq 0, \\ \mathcal{P}_{\text{fullH}}^{\text{norm}}, & \pi = \text{hh},\ \kappa = \text{full_hamiltonian}. \end{cases}$$

Dedup is signature-based (`_polynomial_signature`, `_to_signature`) after normalization/cleaning maps in PAOP section.

Final ADAPT pipeline output map:

$$(\pi, \kappa, H, \psi_{\text{ref}}, \text{controls}) \xrightarrow{\text{pool build}} \mathcal{P}^{(\kappa)} \xrightarrow{\text{8.22 recursion}} (\Theta_{\text{final}}, \mathcal{S}_{\text{final}}, E_{\text{final}}) \xrightarrow{\text{sector exact compare}} (\Delta E,\ |\Delta E|).$$

Payload-level reported scalars are

E_{final}, Θ_{final}, `ansatz_depth` $= |\mathcal{S}_{\text{final}}|$, `pool_size` $= |\mathcal{P}^{(\kappa)}|$, `adapt_inner_optimizer`, `adapt_state_backend`, `stop_reason`, r

9. ADAPT Math and Selection Mechanics

Chapter 8 now contains the full primitive-closed derivation chain for ADAPT/VQE. This chapter keeps only concise addenda that are implementation-indexed and non-duplicative.

6.24 9.1 Cross-reference contract

Use Sections 8.19 to 8.23 as the authoritative substitution ladder for:

- pool-generic ADAPT/VQE symbols,
- objective closure in primitive Pauli action form,
- update recursion and stopping predicate,
- pool construction case split and payload map.

6.25 9.2 Minimal ADAPT recurrence (implementation form)

For direct code matching:

$$\psi_n = \text{Prepare}(\psi_{\text{ref}}, \mathcal{S}_n, \Theta_n), \qquad g_m = 2\,\Im\langle H\psi_n | A_m \psi_n\rangle, \quad m \in \mathcal{I}_n,$$

$$m_n = \arg\max_{m \in \mathcal{I}_n} s_m, \qquad \Theta_{n+1}^{(0)} = (\Theta_n, \theta_{\text{init}}), \qquad \Theta_{n+1}^{\star} \approx \text{SPSA}(E_{n+1}, \Theta_{n+1}^{(0)}; \text{SPSA params}),$$

$$\mathcal{S}_{n+1} = \text{append}(\mathcal{S}_n, A_{m_n}), \qquad E_{n+1} = E_{n+1}(\Theta_{n+1}^{\star}).$$

6.26 9.3 Pool-family addendum

Concrete pool-family definitions remain in:

- Chapter 10 for HH-VA generator structure.
- Chapter 11 for PAOP/PI generator algebra and cleanup/dedup maps.
- Appendix B for compressed substitution templates and worked examples.

In the master ADAPT derivation these are all injected through the single generic symbol $\mathcal{P}^{(\kappa)}$.

6.27 9.4 Code-anchor index for Chapter 8 formulas

- `src/quantum/vqe_latex_python_pairs.py`:
  - `apply_pauli_string`
  - `apply_exp_pauli_polynomial`
  - `expval_pauli_polynomial`
  - `expval_pauli_polynomial_one_apply`
  - `vqe_minimize`
- `pipelines/hardcoded/adapt_pipeline.py`:
  - `_apply_pauli_polynomial`
  - `_commutator_gradient`
  - `_prepare_adapt_state`
  - `_adapt_energy_fn`
  - `_run_hardcoded_adapt_vqe`
- `src/quantum/spsa_optimizer.py`:

  - `spsa_minimize`
- `src/quantum/compiled_polynomial.py`:
  - `compile_polynomial_action`
  - `energy_via_one_apply` # 10. HH-VA (Termwise and Layerwise)

6.28 10.1 HH termwise ansatz primitive

Per layer, termwise HH ansatz applies each single-term Pauli exponential from groups:

1. hopping terms
2. onsite terms
3. potential terms
4. phonon terms
5. electron-phonon terms
6. optional drive terms

with one independent parameter per single-term exponential.

Implementation anchor: `HubbardHolsteinTermwiseAnsatz`.

6.29 10.2 HH layerwise ansatz primitive

Per layer, HH layerwise ansatz keeps same physical groups but shares one parameter per group, while still applying all group terms as ordered exponentials.

Implementation anchor: `HubbardHolsteinLayerwiseAnsatz`.

6.30 10.3 Substitution of grouped generators

For group `G` with term list $\{P_{G,j}\}$:

$$U_G(\theta_G) = \prod_j \exp(-i\theta_G c_{G,j} P_{G,j}).$$

Termwise version substitutes $\theta_{G,j}$ in place of shared θ_G:

$$U_G^{\text{termwise}} = \prod_j \exp(-i\theta_{G,j} c_{G,j} P_{G,j}).$$

10.4 Parameter counting substitution

If group sizes are $M_1, \ldots, M_R$, and reps=`r`:

- layerwise parameter count:

$$N_\theta^{\text{layerwise}} = r \cdot R.$$

- termwise parameter count:

$$N_\theta^{\text{termwise}} = r \cdot \sum_{a=1}^{R} M_a.$$

10.5 Ordering semantics substitution

The unitary ordering implemented is deterministic and follows group order then within-group sorted decomposition.

So final layer unitary is:

$$U_{\text{layer}} = U_{\text{drive}} U_g U_{\text{ph}} U_v U_U U_t$$

or equivalent forward product by code order depending representation convention; implementation applies in explicit loop order stored in term arrays.

6.31 10.6 Final fully substituted HH-VA forms

Layerwise:

$$|\psi\rangle = \left(\prod_{\ell=1}^{r} \prod_{G\in\{t,U,v,ph,g,drive\}} \prod_{j\in G} \exp(-i\theta_{\ell,G} c_{G,j} P_{G,j}) \right) |\psi_{\mathrm{ref}}\rangle.$$

Termwise:

$$|\psi\rangle = \left(\prod_{\ell=1}^{r} \prod_{G} \prod_{j\in G} \exp(-i\theta_{\ell,G,j} c_{G,j} P_{G,j}) \right) |\psi_{\mathrm{ref}}\rangle.$$

11. PAOP/PI Operator Pool Deep Dive

6.32 11.1 Primitive PAOP ingredients

PAOP is built from cached primitives:

- site density n_i
- doublon D_i
- momentum quadrature $P_i = i(b_i^\dagger - b_i)$
- displacement quadrature $X_i = b_i + b_i^\dagger$
- spin-summed hopping $K_{ij} = \sum_\sigma (c_{i\sigma}^\dagger c_{j\sigma} + c_{j\sigma}^\dagger c_{i\sigma})$
- odd hopping/current channel $J_{ij} = i(c_i^\dagger c_j - c_j^\dagger c_i)$ per spin-summed construction in code (`jw_current_hop`).

Implementation anchor: `polaron_paop.py`.

6.33 11.2 Core PI quadrature definition

$$P_i = i(b_i^\dagger - b_i).$$

Implementation substitution:

$$P_i \leftarrow (1j)\, b_i^\dagger + (-1j)\, b_i.$$

The builder ensures real coefficients after cleaning when required by selected options.

6.34 11.3 Shifted density substitution

$$\tilde{n}_i = n_i - \bar{n} I, \qquad \bar{n} = \frac{N_\uparrow + N_\downarrow}{L} \text{ (default fallback 1 if needed).}$$

11.4 PAOP family generators

6.34.1 11.4.1 Local conditional displacement dressing

$$G_i^{\mathrm{disp}} = \tilde{n}_i P_i.$$

11.4.2 Local doublon dressing

$$G_i^{\mathrm{dbl}} = \tilde{n}_i D_i.$$

11.4.3 Dressed hopping drag

$$G_{ij}^{\mathrm{hopdrag}} = K_{ij}(P_i - P_j).$$

11.4.4 Odd-channel drag

$$G_{ij}^{\mathrm{curdrag}} = J_{ij}(P_i - P_j).$$

11.4.5 Second-order even channel

$$G_{ij}^{\mathrm{hop2}} = K_{ij}(P_i - P_j)^2.$$

Optional term dropping can remove terms with phonon identity support from this channel.

6.34.2 11.4.6 Extended cloud channels

For radius-`R` site pairs:

$$G_{i\to j}^{\text{cloud_p}} = \tilde{n}_i P_j, \qquad G_{i\to j}^{\text{cloud_x}} = \tilde{n}_i X_j.$$

11.4.7 Doublon translation channels

$$G_{i\to j}^{\text{dbl_p}} = D_i P_j, \qquad G_{i\to j}^{\text{dbl_x}} = D_i X_j.$$

11.5 Pool-key-to-content substitution

Implementation mapping:

- `paop_min`: displacement only.
- `paop_std`: displacement plus hopdrag.
- `paop_full`: displacement plus doublon plus hopdrag plus extended cloud (`R>=1`).
- `paop_lf_std`: `paop_std` plus curdrag.
- `paop_lf2_std`: `paop_lf_std` plus hop2.
- `paop_lf_full`: `paop_lf2_std` plus extended cloud and doublon-translation channels.

Aliases:

- `paop -> paop_std`
- `paop_lf -> paop_lf_std`

6.35 11.6 Cleaning, normalization, split, dedup substitution

Given polynomial $G = \sum_k c_k P_k$:

1. normalization options:

$$G \to G \quad \text{(none)}, \qquad G \to \frac{G}{\|G\|_F} \quad \text{(fro)}, \qquad G \to \frac{G}{\max_k |c_k|} \quad \text{(maxcoeff)}.$$

2. cleaning then drops terms with $|c_k| \le \epsilon$, and checks imaginary tolerance.
3. split mode converts each nonzero term into individual generator.
4. dedup uses sorted signature tuple $(\text{word}, \text{round}(\text{Re}(c), 12))$.

6.36 11.7 Final fully substituted PAOP/PI operator-pool statement

$$\mathcal{P}_{\text{PAOP}}(\text{mode}) = \text{Dedup}\left(\text{Split}\left(\text{Clean}\left(\text{Normalize}\left(\{G_a(\tilde{n}, D, K, J, P, X)\}_a\right)\right)\right)\right)$$

with `mode` selecting the enabled generator family subset as listed above.

7 12. Appendix A: Code-Anchor Mathematical Ledger

7.1 12.1 HH construction ledger

- `build_hubbard_kinetic`: nearest-neighbor, spin-resolved hopping assembly.
- `build_hubbard_onsite`: onsite product of number operators.
- `build_hubbard_potential`: static local potential.
- `build_holstein_phonon_energy`: $\omega_0 \sum_i (n_{b,i} + 1/2)$.
- `build_holstein_coupling`: $g \sum_i x_i (n_i - I)$.
- `build_hubbard_holstein_drive`: $\sum_{i,\sigma} (v_i(t) - v_{0,i}) n_{i\sigma}$ routed via potential sign convention.
- `build_hubbard_holstein_hamiltonian`: full sum.

7.2 12.2 VQE and exponential ledger

- `apply_pauli_string`: exact action with rightmost-qubit convention.
- `apply_pauli_rotation`: single-Pauli closed form.
- `apply_exp_pauli_polynomial`: ordered product over terms.
- `expval_pauli_polynomial`: expectation accumulation.

- `expval_pauli_polynomial_one_apply`: compiled one-apply energy branch.
- `vqe_minimize`: multi-restart optimization front-end.

7.3 12.3 ADAPT ledger

- `_build_uccsd_pool`, `_build_cse_pool`, `_build_full_hamiltonian_pool`, `_build_hva_pool`, `_build_paop_pool`.
- `_commutator_gradient`: analytic commutator gradient implementation.
- `_run_hardcoded_adapt_vqe`: full ADAPT loop with fallback and HH seed preconditioning.
- `spsa_minimize`: SPSA inner optimizer branch for VQE and ADAPT reoptimization.

7.4 12.4 PAOP ledger

- `_make_paop_core`: generator family construction.
- `jw_current_hop`: odd channel JW form. # 13. Appendix B: Substitution Compression (Implementation-Aligned)

This appendix replaces the previous one-by-index long list with one template law plus two examples.

7.5 B.1 Canonical PAOP templates

Define shifted density and canonical primitives:

$$\tilde{n}_i := n_i - \bar{n}\, I, \qquad n_i = n_{i\uparrow} + n_{i\downarrow}, \qquad \bar{n} = \frac{N_\uparrow + N_\downarrow}{L},$$

$$P_i = i(\hat{b}_i^\dagger - \hat{b}_i), \qquad X_i = \hat{b}_i + \hat{b}_i^\dagger, \qquad D_i = n_{i\uparrow} n_{i\downarrow},$$

$$K_{ij} = \sum_{\sigma\in\{\uparrow,\downarrow\}} (\hat{c}_{i\sigma}^\dagger \hat{c}_{j\sigma} + \hat{c}_{j\sigma}^\dagger \hat{c}_{i\sigma}), \qquad J_{ij} = i \sum_{\sigma\in\{\uparrow,\downarrow\}} (\hat{c}_{i\sigma}^\dagger \hat{c}_{j\sigma} - \hat{c}_{j\sigma}^\dagger \hat{c}_{i\sigma}).$$

Template family (before index substitution):

$$\mathcal{G}(\rho) = \begin{cases} \mathcal{G}_i^{\text{disp}} = \tilde{n}_i P_i, & \rho = (i, \text{disp}), \\ \mathcal{G}_i^{\text{dbl}} = \tilde{n}_i D_i, & \rho = (i, \text{dbl}), \\ \mathcal{G}_{ij}^{\text{hopdrag}} = K_{ij}(P_i - P_j), & \rho = (i, j, \text{hopdrag}), \\ \mathcal{G}_{ij}^{\text{curdrag}} = J_{ij}(P_i - P_j), & \rho = (i, j, \text{curdrag}), \\ \mathcal{G}_{ij}^{\text{hop2}} = K_{ij}(P_i - P_j)^2, & \rho = (i, j, \text{hop2}), \\ \mathcal{G}_{i\to j}^{\text{cloud}_p} = \tilde{n}_i P_j,\ \mathcal{G}_{i\to j}^{\text{cloud}_x} = \tilde{n}_i X_j, & \rho = (i, j, \text{cloud}), \\ \mathcal{G}_{i\to j}^{\text{dbl}_p} = D_i P_j,\ \mathcal{G}_{i\to j}^{\text{dbl}_x} = D_i X_j, & \rho = (i, j, \text{dbl trans}). \end{cases}$$

For any ρ, code applies:

$$\mathcal{F}_{\text{site}} : \rho \mapsto (i, j, \text{mode}, \text{coeff}, \text{radius}), \qquad \mathcal{F}_{\text{op}} : \mathcal{G} \mapsto \sum_r c_r\, Q_r,\ Q_r \in \{e, x, y, z\}^{\otimes N_q},$$

then cleanup and dedup:

$$\mathcal{F}_{\text{clean}} : \sum_r c_r Q_r \mapsto \sum_{r\in\mathcal{K}} c_r' Q_r, \quad \mathcal{F}_{\text{dedup}} : \text{sort+signature}.$$

Implementation anchors: - `mode_index`, `jw_number_operator`, `boson_operator`, `boson_displacement_operator`, - `fermion_plus_operator`, `fermion_minus_operator`, `jw_current_hop`, `PauliPolynomial` arithmetic, - `_normalize_poly`, `_clean_poly`, `_append_operator`, `_to_signature`.

7.6 B.2 Worked example A: nearest-neighbor hopdrag

Primitive:

$$\mathcal{O}_A = \tilde{n}_0 P_0 + \eta\, K_{01}(P_0 - P_1).$$

Substitute density and operator blocks:

$$\tilde{n}_0 = (1-\bar{n})\, I - \frac{1}{2}\left(Z_{p(0,\uparrow)} + Z_{p(0,\downarrow)}\right), \qquad P_r = i\left(\hat{b}_r^\dagger - \hat{b}_r\right),$$

$$\mathcal{O}_A = \left((1-\bar{n})\, I - \frac{1}{2}\left(Z_{p(0,\uparrow)} + Z_{p(0,\downarrow)}\right)\right) P_0 + \eta\, K_{01}(P_0 - P_1),$$

followed by

$$\mathcal{F}_{\text{site}},\ \mathcal{F}_{\text{op}},\ \mathcal{F}_{\text{clean}},\ \mathcal{F}_{\text{dedup}}.$$

The algebra does not change for every other nearest-neighbor entry pair; only (i, j, ρ) changes.

7.7 B.3 Worked example B: periodic edge hopdrag

For periodic boundary hop $(L-1) \to 0$:

$$\mathcal{O}_B = \tilde{n}_{L-1} P_{L-1} + \eta\, K_{L-1,0}(P_0 - P_{L-1}),$$

with the same substitution map as B.2. This is exactly the same operator-valued formula with a different index map produced by `_distance_1d` + `bravais_nearest_neighbor_edges` and a nonzero `periodic` flag.

7.8 B.4 Signatures and dedup

Given cleaned reduced polynomial $G = \sum_r c_r Q_r$, signature is

$$\Sigma(G) = \operatorname{sort}\big((Q_r, \operatorname{round}(\Re c_r, 12))\big)_{r:\,|c_r|>\epsilon}.$$

Duplicates are removed by insertion order in `_to_signature`, matching `polaron_paop.py`.

7.9 B.5 Final compact closure

For all PAOP blocks the entire chain is:

$$\mathcal{G} \xrightarrow{\mathcal{F}_{\text{site}}} \mathcal{G}(i, j, \dots) \xrightarrow{\mathcal{F}_{\text{op}}} \sum_r c_r Q_r \xrightarrow{\mathcal{F}_{\text{clean}}} \sum_{r\in\mathcal{K}} \tilde{c}_r Q_r \xrightarrow{\mathcal{F}_{\text{dedup}}} \mathcal{G}_{\text{canonical}}.$$

Hence the earlier 50+ index-shift blocks collapse into one template and two examples.

8 14. Appendix C: End-to-End Substitution Chain (Drive, Sector, Pool)

8.1 C.1 Variational functional and ansatz

$$\mathcal{F}(t, \Theta) = \langle \psi(\Theta) | H_{\text{HH}}(t) | \psi(\Theta) \rangle, \qquad |\psi(\Theta)\rangle = \left(\prod_{k=1}^{n} e^{-i\theta_k A_k}\right) |\psi_{\text{ref}}\rangle.$$

8.2 C.2 Hamiltonian decomposition after all substitutions

$$H_{\text{HH}}(t) = H_t + H_U + H_v + H_{\text{ph}} + H_g + H_{\text{drive}}(t), \qquad H_{\text{drive}}(t) = \sum_{i,\sigma} (v_i(t) - v_{0,i}) n_{i\sigma}.$$

8.3 C.3 Primitive substitutions

$$n_{i\sigma} = \frac{I - Z_{p(i,\sigma)}}{2}, \quad n_i = n_{i\uparrow} + n_{i\downarrow}, \quad X_i = \hat{b}_i + \hat{b}_i^\dagger,$$

$$H_g = g \sum_i X_i \left(n_i - I\right) = -\frac{g}{2} \sum_i X_i \big(Z_{p(i,\uparrow)} + Z_{p(i,\downarrow)}\big),$$

$$H_{\text{drive}}(t) = \frac{1}{2} \sum_{i,\sigma} (v_i(t) - v_{0,i}) I - \frac{1}{2} \sum_{i,\sigma} (v_i(t) - v_{0,i}) Z_{p(i,\sigma)}.$$

8.4 C.4 ADAPT coupling and exact-sector target

For pool entry A_m,

$$g_m = i\langle\psi|[H_{\text{HH}}(t), A_m]|\psi\rangle, \qquad g_m = 2\ \text{Im}\langle H_{\text{HH}}(t)\psi|A_m\psi\rangle.$$

Exact comparison energy:

$$E_{\text{exact}} = \begin{cases} \lambda_{\min}\left(\Pi^{\text{fermion}}_{N_\uparrow,N_\downarrow} H_{\text{HH}}\right), & \text{HH} \\ \lambda_{\min}\left(\Pi^{\text{full}}_{N_\uparrow,N_\downarrow} H_{\text{Hub}}\right), & \text{Hubbard} \end{cases}$$

and $\Delta E = E_{\text{var}} - E_{\text{exact}}$.

8.5 C.5 Final chain closure

$$|\psi_{\text{out}}\rangle = \left(\prod_{k=1}^{n} e^{-i\theta_k A_k}\right) |\psi_{\text{ref}}\rangle, \qquad \left(H_{\text{HH}}(t) = \sum_r h_r(t) Q_r\right) \text{ in } Q_r \in \{e, x, y, z\}^{\otimes N_q},$$

with selected generators $A_k \in \mathcal{P}_{\text{selected}} \subseteq \mathcal{P}_{\text{HH–VA}} \cup \mathcal{P}_{\text{PAOP}} \cup \mathcal{P}_{\text{ADAPT}}$.

9 15. Appendix D: Alternate Inner Optimizer and Backend Branches

9.1 D.1 COBYLA alternate branch

When `adapt_inner_optimizer=COBYLA`, inner refit in `_run_hardcoded_adapt_vqe` is:

$$\Theta^\star_{n+1} \approx \underset{\Theta \in \mathbb{R}^{n+1}}{\arg\min}\, E_{n+1}(\Theta) \quad \text{via} \quad \texttt{minimize}(_obj, \Theta^{(0)}_{n+1}, \texttt{method} = \text{COBYLA}, \texttt{options} = \{\texttt{maxiter}, \texttt{rhobeg} = 0.3\}).$$

HH seed preconditioning branch under COBYLA is

$$\phi_\star \approx \underset{\phi}{\arg\min}\, E_{\text{seed}}(\phi) \quad \text{with} \quad \texttt{maxiter} = \max(100, \min(\texttt{adapt_maxiter}, 600)).$$

This branch keeps the same pool growth, gradient selection, fallback, and stopping predicates as Chapter 8. Code anchors: `pipelines/hardcoded/adapt_pipeline.py`, `src/quantum/vqe_latex_python_pairs.py`.

9.2 D.2 Compiled one-apply backend branch

When compiled backend is enabled (`vqe_energy_backend=one_apply_compiled` or `adapt_state_backend=compiled`), the core primitive is:

$$\widetilde{H} = \text{Compile}(H), \qquad h_\psi = \widetilde{H}|\psi\rangle, \qquad E(\psi) = \langle\psi|h_\psi\rangle.$$

So energy evaluation is a one-apply contraction:

$$E(\Theta) = \langle\psi(\Theta)|\, \widetilde{H}\, |\psi(\Theta)\rangle\,.$$

Compiled ADAPT commutator form uses

$$g_m = 2\,\Im\langle h_\psi|a^{(m)}_\psi\rangle, \qquad a^{(m)}_\psi = \widetilde{A}_m|\psi\rangle,$$

where $\widetilde{A}_m$ is compiled from pool polynomial A_m.

Compiled ansatz execution branch:

$$|\psi(\Theta)\rangle = \text{CompiledPrepare}(\Theta, |\psi_{\text{ref}}\rangle, \mathcal{S}),$$

then

$$E(\Theta) = \text{OneApplyEnergy}(\widetilde{H}, |\psi(\Theta)\rangle).$$

Code anchors:

- `src/quantum/compiled_polynomial.py`
- `src/quantum/compiled_ansatz.py`
- `src/quantum/vqe_latex_python_pairs.py` (`expval_pauli_polynomial_one_apply`)
- `pipelines/hardcoded/adapt_pipeline.py` (`energy_via_one_apply`, compiled-state branch)

9.3 D.3 Optimizer/backend switch map

The implementation branch map is:

$$\texttt{adapt_inner_optimizer} \in \{\text{SPSA}, \text{COBYLA}\}, \qquad \texttt{adapt_state_backend} \in \{\text{compiled}, \text{legacy}\},$$

$$\texttt{vqe_method} \in \{\text{SPSA}, \text{SciPy methods}\}, \qquad \texttt{vqe_energy_backend} \in \{\text{one_apply_compiled}, \text{legacy}\}.$$

SPSA-main derivation in Chapter 8 is unchanged by these switches; only inner optimizer mechanics and objective-evaluation backend are swapped.